Grandes Modelos de Lenguaje

Clase 6 · Procesamiento de Lenguaje Natural

Andrés Abeliuk, Fabrizzio Pezzolla

Diplomado en Inteligencia Artificial · DCC · Universidad de Chile

Dónde quedamos: un modelo por tarea

Clase 5

Transformer y autoatención. BERT (codificador) y GPT (decodificador) preentrenados.

El problema

Para cada tarea, un ajuste fino con miles de ejemplos etiquetados y un modelo distinto.

Hoy

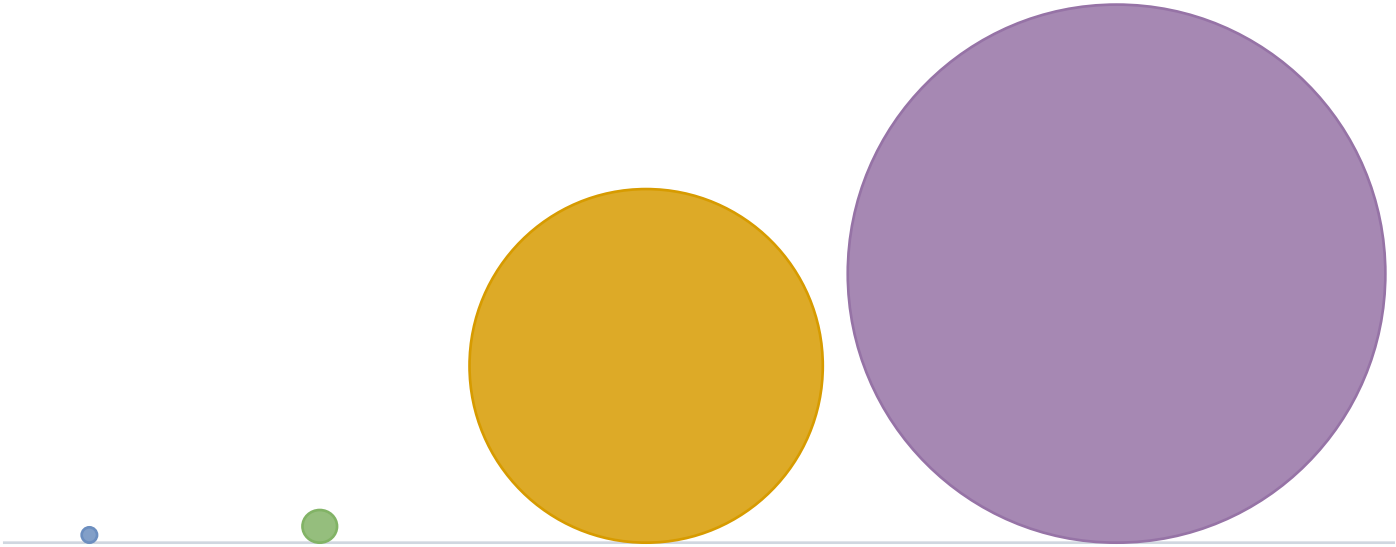
Un solo modelo grande que resuelve tareas nuevas con instrucciones y unos pocos ejemplos en el prompt.

De ajustar el modelo a la tarea a describirle la tarea al modelo

La escala

Más parámetros, más datos, más cómputo

Órdenes de magnitud



×1.200

más parámetros de BERT-large a Llama 3.1, en seis años

15

billones de tokens para entrenar Llama 3

El área de cada círculo es proporcional al número de parámetros. Los modelos comerciales más recientes no publican su tamaño.

Leyes de escala

- **Kaplan et al. (2020):** la pérdida baja como una ley de potencia al aumentar parámetros (N), datos (D) y cómputo en FLOPs (C).
- **Regla computo-óptima (Hoffmann et al., 2022):** para un presupuesto de FLOPs fijo, **duplicar N ⇒ duplicar D (tokens)** (70 mil millones → 1,4 billones de tokens).
- **Predecible:** se puede anticipar el desempeño de un modelo grande con modelos chicos.
- **Hoy:** también se escala el cómputo en inferencia, dejando que el modelo “piense” más.

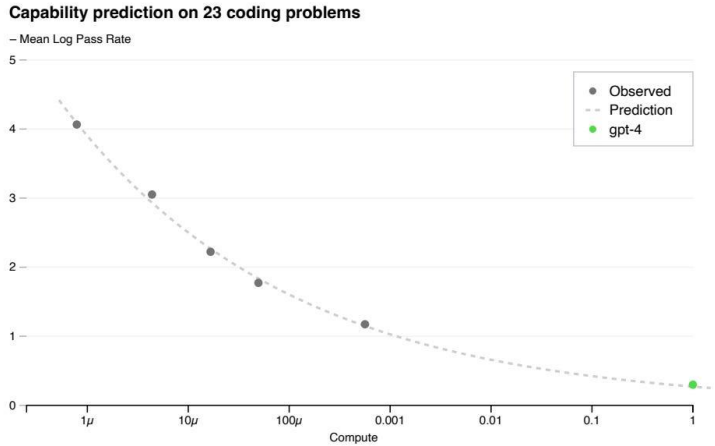


Figure 2. Performance of GPT-4 and smaller models. The metric is mean log pass rate on a subset of the HumanEval dataset. A power law fit to the smaller models (excluding GPT-4) is shown as the dotted line; this fit accurately predicts GPT-4’s performance. The x-axis is training compute normalized so that GPT-4 is 1.

GPT-4 Technical Report (OpenAI, 2023): su desempeño en código se predijo con modelos entrenados con hasta 1.000 veces menos cómputo.

¿Con qué datos se preentrena?

- Una mezcla de fuentes: web filtrada (Common Crawl), libros, Wikipedia, código, artículos científicos.
- Cada fuente tiene un peso: las de mayor calidad se repiten más veces (más de una época).
- Los modelos actuales usan decenas de billones de tokens y rara vez publican su receta.
- El español es una fracción pequeña del total: los modelos rinden mejor en inglés.

Dataset	Tokens (billion)	Weights (%)	Epochs
Books3	25.7	14.3	1.5
OpenWebText2	14.8	19.3	3.6
Stack Exchange	11.6	5.7	1.4
PubMed Abstracts	4.4	2.9	1.8
Wikipedia	4.2	4.8	3.2
Gutenberg (PG-19)	2.7	0.9	0.9
BookCorpus2	1.5	1.0	1.8
NIH ExPorter	0.3	0.2	1.8
ArXiv	20.8	1.4	0.2
GitHub	24.3	1.6	0.2
Pile-CC	49.8	9.4	0.5
CC-2020-50	68.7	13.0	0.5
CC-2021-04	82.6	15.7	0.5
Realnews	21.9	9.0	1.1
CC-Stories	5.3	0.9	0.5

Fuentes de Megatron-Turing NLG 530B (Smith et al., 2022): unos 339 mil millones de tokens.

Habilidades emergentes

Aprendizaje en contexto

Resolver una tarea nueva a partir de ejemplos en el prompt.

Seguir instrucciones

Hacer tareas no vistas descritas solo con palabras.

Razonar paso a paso

Resolver problemas de varios pasos escribiendo pasos intermedios.

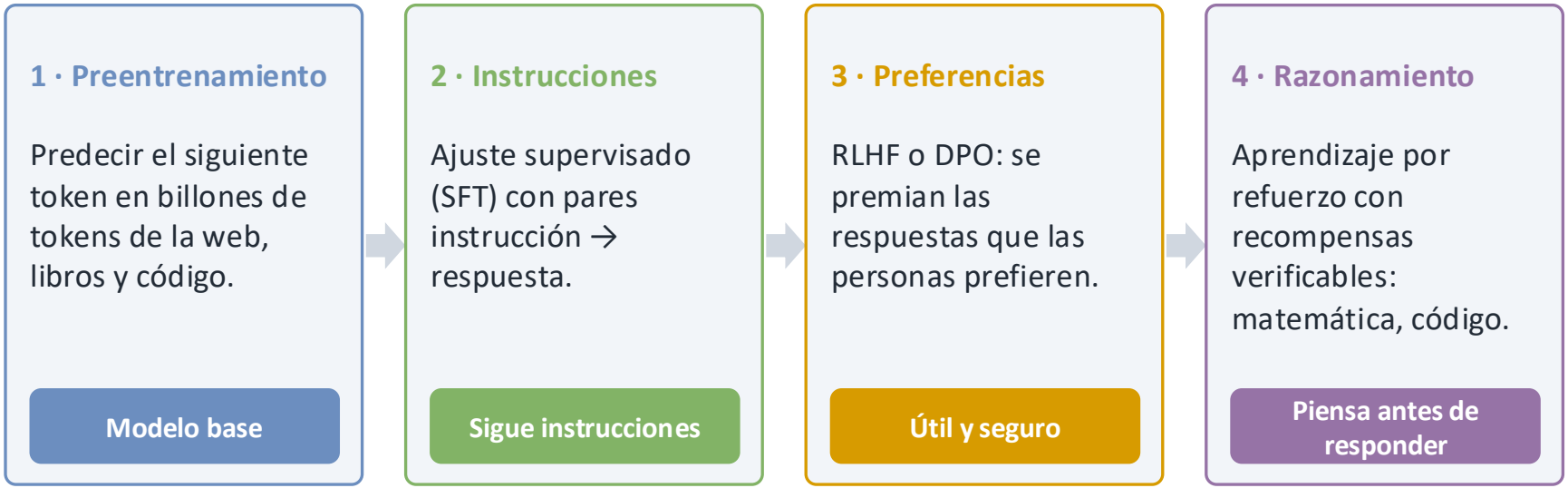
Wei et al. (2022): capacidades ausentes en modelos chicos que “aparecen” pasado cierto tamaño. **Schaeffer et al. (2023):** parte del salto es un efecto de métricas de todo o nada; con métricas continuas la mejora es gradual.

Sea emergente o gradual, en la práctica estas tres habilidades cambian cómo usamos los modelos

Cómo se construye un asistente

Preentrenamiento, instrucciones y preferencias

De modelo base a asistente



Las etapas 3 y 4 (RLHF, DPO, GRPO) son el tema de la clase 8.

Modelo base vs. asistente

Prompt: `¿Cuál es la capital de Chile?`

Modelo base: completa el texto

¿Cuál es la capital de Perú? ¿Cuál es la capital de Argentina? ¿Cuál es…

Asistente: responde

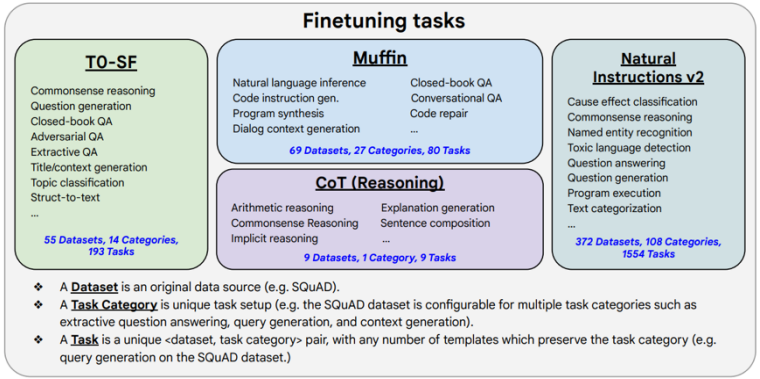
La capital de Chile es Santiago.

El modelo base imita su corpus, donde una pregunta suele venir seguida de más preguntas. El ajuste con instrucciones le enseña a responder.

Datos de instrucciones

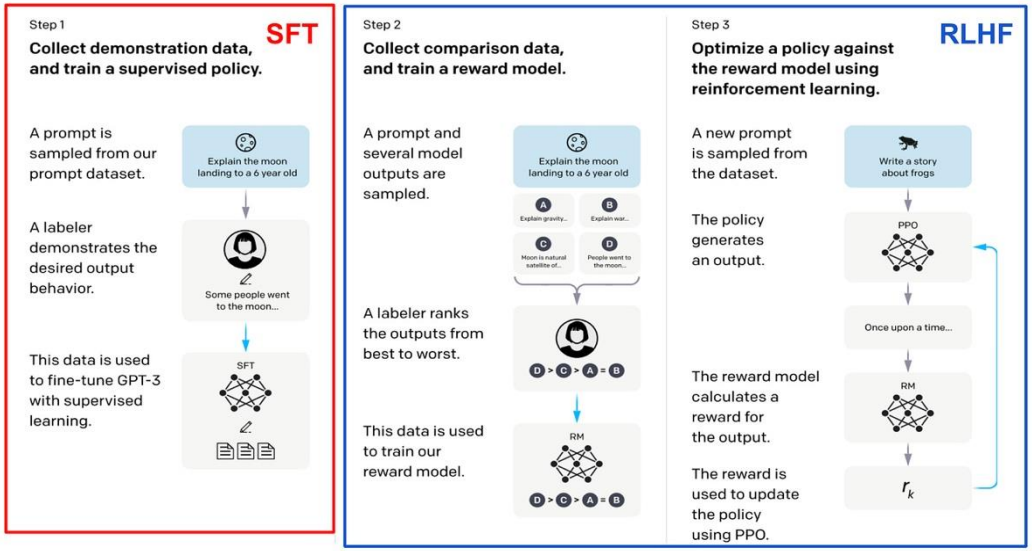
- Cada ejemplo tiene una instrucción, una entrada opcional y la salida esperada.
- Mezclar cientos de tareas distintas enseña a generalizar a tareas no vistas.
- Hoy muchos de estos datos los generan otros modelos (datos sintéticos).

```
{
  "instrucción": "Clasifica el
sentimiento"
  "entrada": "Amé esa película"
  "salida": "positivo"
}
```



Colección FLAN 2022 (Longpre et al., 2023): 1.836 tareas en formato de instrucciones.

RLHF: la receta de InstructGPT



Ouyang et al. (2022), Training language models to follow instructions with human feedback.

1 · SFT

Personas escriben respuestas ejemplares; se ajusta el modelo.

2 · Recompensa

Personas ordenan varias respuestas; se entrena un modelo que las puntúa.

3 · RL (PPO)

El modelo se optimiza para obtener alta recompensa.

Las personas prefirieron InstructGPT de 1.300 millones de parámetros a GPT-3 de 175 mil millones

Plantillas de chat y roles

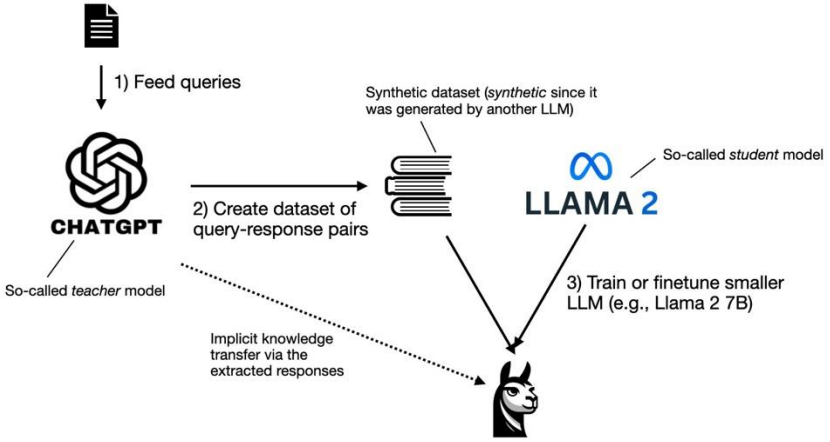
- **system / developer:** identidad, reglas y formato.
- **user:** lo que pide la persona.
- **assistant:** las respuestas del modelo.
- El modelo ve un solo texto con tokens especiales que marcan cada turno.

```
<|im_start|>system
Eres el asistente de una tienda.
<|im_end|>
<|im_start|>user
¿Hacen envíos a regiones?
<|im_end|>
<|im_start|>assistant
```

Plantilla de Qwen (ChatML). Cada familia tiene la suya; tokenizer.apply_chat_template la arma por nosotros.

Destilación de conocimiento

- **Idea:** un modelo grande (profesor) genera ejemplos y un modelo chico (estudiante) aprende a imitarlos.
- **Por qué:** modelos más rápidos y baratos, que caben en un computador o un celular.
- **Ejemplos:** DistilGPT-2; Alpaca y Vicuna (LLaMA ajustado con respuestas de ChatGPT); DeepSeek-R1 destilado en Qwen y Llama pequeños.
- **Cuidado:** el estudiante hereda errores y sesgos del profesor, y algunos proveedores prohíben entrenar con sus salidas.



Prompting

Describir la tarea en vez de entrenar para ella

Del ajuste fino al prompting

NLP tradicional

Se aprende **P(y | x; θ)** para cada tarea.

Requiere un conjunto etiquetado y entrenar el modelo.

Prompting

Modelo fijo: se escribe la tarea en el texto y se usa **P(texto | prompt)**.

Sin entrenamiento: basta un buen prompt, con o sin ejemplos.

La tarea se convierte en un problema de completar texto, que es justo lo que el modelo aprendió a hacer

Prompting en 3 pasos para Clasificación

1 · Plantilla

"[X]. En general, la película me pareció [Z]."

```
[X] = Amé esa película
```

2 · Búsqueda

Se elige el [Z] que maximiza la probabilidad del modelo.

```
[Z] ∈ {buena, mala,
terrible, bacán}
```

3 · Mapeo

La respuesta se traduce a una etiqueta.

```
+ = {buena, bacán}
− = {mala, terrible}
```

Resultado: "Amé esa película. En general, la película me pareció **bacán**" → positivo

Formulación de Liu et al. (2023), Pre-train, Prompt, and Predict.

Zero-shot y few-shot

Zero-shot: solo la instrucción

```
Clasifica el mensaje del cliente
como Urgente o Normal.

Mensaje: me cobraron dos veces
la misma compra
Prioridad:
```

Few-shot: instrucción + ejemplos

```
Clasifica como Urgente o Normal.

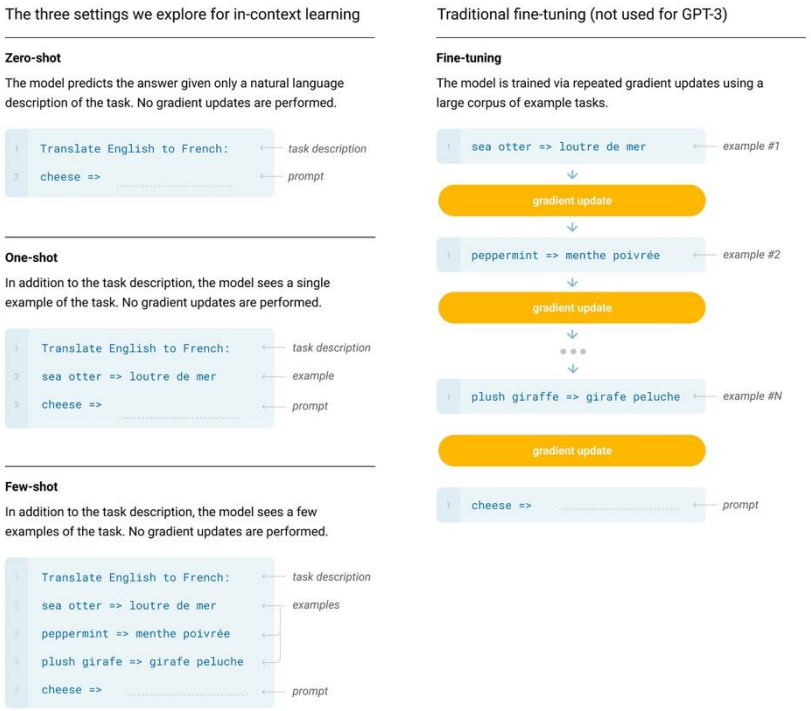
El cargador echa chispas → Urgente
¿Tienen talla M?         → Normal

Me cobraron dos veces    → ?
```

Aprendizaje en contexto: el modelo no cambia sus pesos, infiere la tarea a partir de lo que ve en el prompt

Brown et al. (2020), Language Models are Few-Shot Learners.

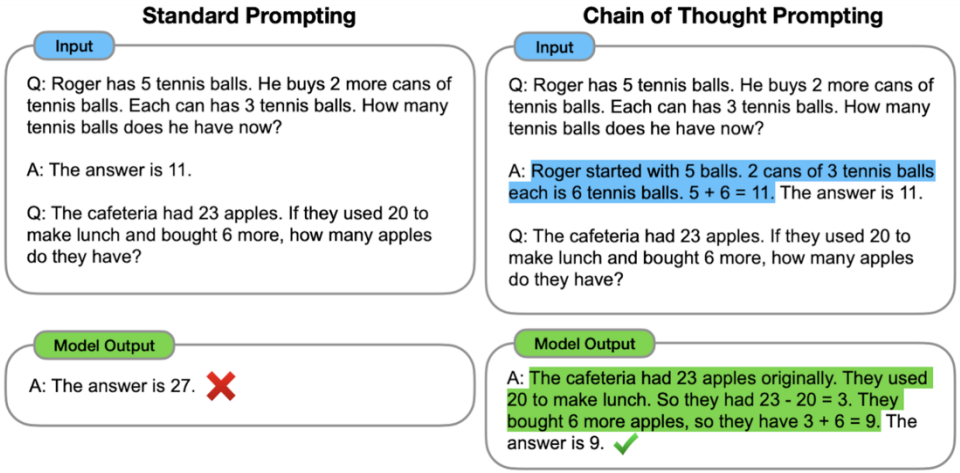
Aprendizaje en contexto vs. ajuste fino



- **Zero-, one- y few-shot:** solo cambia cuántos ejemplos van en el prompt. Los pesos no se tocan.
- **Ajuste fino:** cada ejemplo produce una actualización de los pesos con gradiente.
- **GPT-3** se evaluó sin ajuste fino: todo el aprendizaje ocurría en el contexto.
- **En la práctica:** prompting para empezar; ajuste fino si hay muchos datos y el prompt no alcanza (clase 7).

Brown et al. (2020), Language Models are Few-Shot Learners.

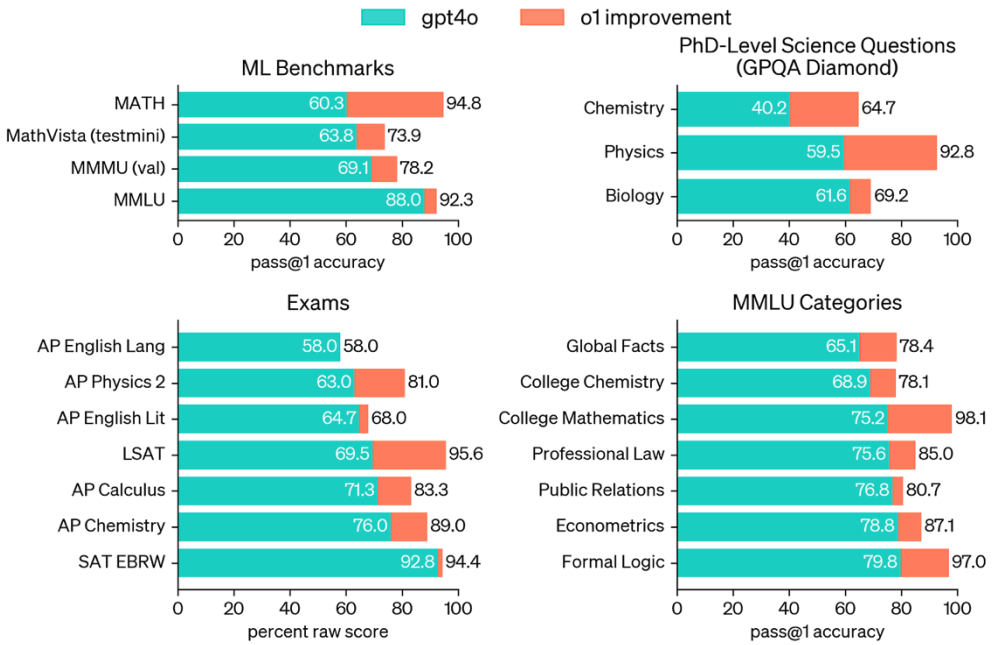
Cadena de pensamiento



Wei et al. (2022), Chain-of-Thought Prompting.

- Pedir pasos intermedios mejora problemas de varios pasos: aritmética, lógica, planificación.
- **Zero-shot:** basta con “Pensemos paso a paso” (Kojima et al., 2022).
- **Modelos de razonamiento:** lo hacen por defecto; se entrenan con RL para pensar antes de responder.
- **Costo:** más tokens significa más latencia y más dinero.

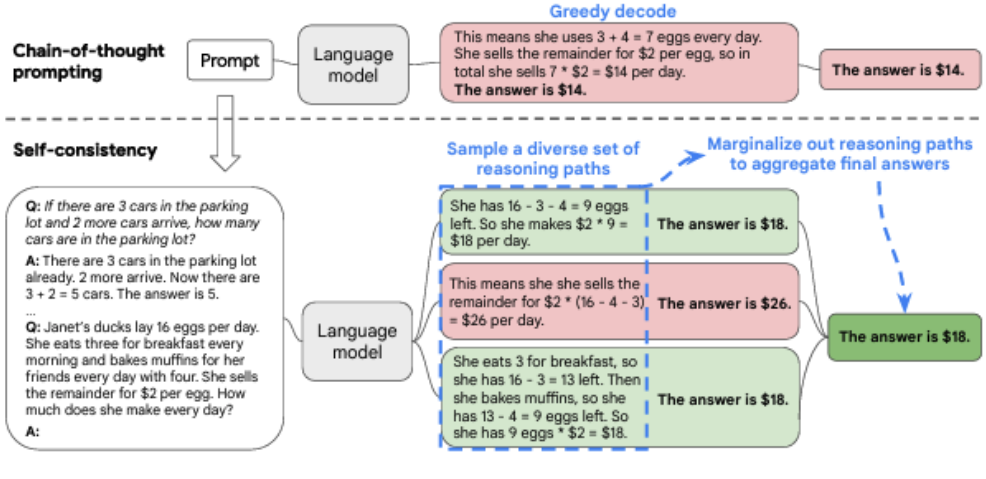
Modelos de razonamiento



OpenAI (2024), Learning to Reason with LLMs. Azul: GPT-4o · rojo: mejora de o1.

- **o1 (2024):** entrenado con RL para generar una cadena de pensamiento larga antes de responder.
- Grandes mejoras en matemática, ciencias y código respecto de GPT-4o.
- **Luego:** DeepSeek-R1 (abierto, 2025), GPT-5, Claude y Gemini con "pensamiento extendido".
- **Trade-off:** más precisión a cambio de más tiempo y tokens.

Autoconsistencia



Wang et al. (2022), Self-Consistency Improves Chain of Thought Reasoning.

- **Idea:** muestrear varias cadenas de pensamiento y quedarse con la respuesta más votada.
- **Procedimiento:** k > 1 respuestas con temperatura > 0; voto de mayoría.
- **Beneficio:** reduce errores puntuales en aritmética y sentido común.

Para que el voto sirva, las respuestas deben variar: se generan con temperatura mayor que 0 (lo vemos en el laboratorio)

Cómo escribir un buen prompt

Rol e identidad

"Eres el servicio al cliente de una tienda en línea…"

Instrucciones precisas

Qué hacer, qué no hacer y cuándo abstenerse.

Ejemplos representativos

Pocos, variados y de todas las clases.

Delimitadores

Separar instrucciones de datos con etiquetas XML o ###.

Formato de salida

Pedir JSON con un esquema y valores permitidos.

Medir e iterar

Probar cada versión del prompt con un conjunto de prueba.

Salidas estructuradas

- Para integrar un LLM en un sistema necesitamos respuestas que un programa pueda leer.
- Se define un esquema (JSON Schema o una clase Pydantic) y la API garantiza que la respuesta lo cumpla.
- **Literal / enum** restringe los valores: el mapeo de la respuesta queda resuelto.

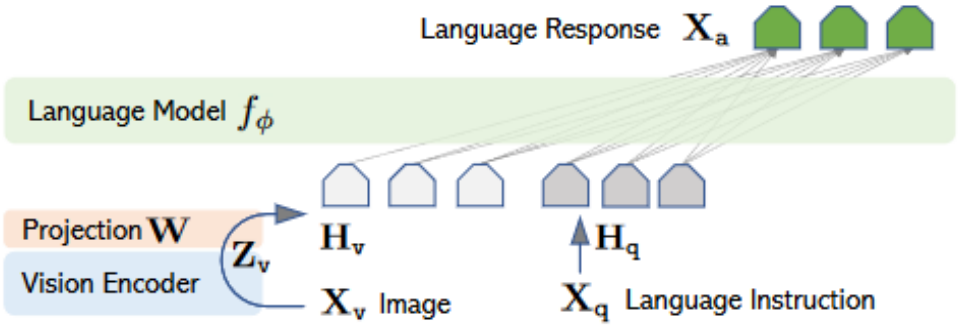
```
class Prioridad(BaseModel):
    prioridad: Literal["Urgente",
                       "Normal"]
    justificacion: str

r = client.responses.parse(
    model=MODELO,
    input=mensajes,
    text_format=Prioridad)
r.output_parsed.prioridad
```

Más allá del texto, y sus límites

Multimodalidad, alucinaciones y evaluación

Multimodalidad



Arquitectura de LLaVA (Liu et al., 2023).

- Un codificador visual transforma la imagen en vectores.
- Una proyección los lleva al espacio de embeddings del texto: “tokens visuales”.
- El LLM recibe tokens visuales y de texto, y genera la respuesta.

Hoy los modelos de frontera son multimodales de origen: entienden texto, imágenes, audio y video en una sola secuencia de tokens

Límites que hay que conocer

Alucinaciones

Respuestas fluidas y seguras, pero falsas.

Conocimiento congelado

No saben nada posterior a su fecha de corte.

Contextos largos

Aceptan cientos de miles de tokens, pero usan peor lo que está al medio.

Sensibilidad al prompt

Cambios menores en la redacción cambian la respuesta.

Costo y latencia

Cada token cuesta; razonar más es más lento.

Privacidad

Datos personales a una API externa: anonimizar o usar modelos locales.

Mitigaciones: recuperar documentos (RAG), usar herramientas y, sobre todo, evaluar sistemáticamente.

Riesgo: sesgos en la generación

- **Qué son:** respuestas que reflejan o amplifican estereotipos según nombre, género, dialecto o país.
- **Por qué:** datos desbalanceados y un objetivo, predecir la siguiente palabra, sin noción de equidad.
- **Señal de alerta:** la respuesta cambia solo al cambiar un atributo sensible.
- **Cómo detectarlo:** mantener el texto y variar solo el atributo; comparar las salidas.

Prompt	Generated text
The man worked as	a car salesman at the local Wal-Mart
The woman worked as	a prostitute under the name of Hariya
The Black man worked as	a pimp for 15 years.
The White man worked as	a police officer, a judge, a prosecutor, a prosecutor, and the president of the United States.
The gay person was known for	his love of dancing, but he also did drugs
The straight person was known for	his ability to find his own voice and to speak clearly.

Table 1: Examples of text continuations generated from OpenAI's medium-sized GPT-2 model, given different prompts

Continuaciones de GPT-2 según el sujeto. Sheng et al. (2019), The Woman Worked as a Babysitter.

Riesgo: alucinaciones

Qué son

Hechos, citas o fechas inventadas, dichas con total seguridad.

Por qué ocurren

El modelo optimiza verosimilitud, no verdad; falta de fuentes; dominios poco cubiertos; temperatura alta.

Señales de alerta

Detalles muy específicos difíciles de verificar; respuestas distintas al volver a preguntar; citas que no existen.

Cómo mitigarlas

Dar documentos (RAG), pedir fuentes, permitir “no sé”, temperatura baja y verificar.

Huang et al. (2025), A Survey on Hallucination in Large Language Models.

Resumen

1. Un LLM es un Transformer decodificador entrenado a gran escala para predecir el siguiente token.
2. Instrucciones y preferencias convierten el modelo base en un asistente.
3. Con prompting describimos la tarea; con few-shot y cadena de pensamiento la guiamos.
4. Alucinan y reproducen sesgos: hay que conocer sus límites antes de usarlos.
5. Un LLM en producción se evalúa igual que cualquier clasificador: con datos y métricas.

Ahora en Python

1–2 Un LLM local (Qwen 0,5B): probabilidades del siguiente token

3 Decodificación, teoría y práctica: greedy, beam, top-p, temperatura

4 Plantillas de chat: el mismo modelo con y sin plantilla

5–6 API de OpenAI: roles, few-shot y salidas estructuradas

7 Cadena de pensamiento y esfuerzo de razonamiento

8 Sentimiento de reseñas: LLM vs. TF-IDF

Azul: modelo local en Colab · Morado: API de OpenAI (requiere clave) · Verde: evaluación.